

INTELLIGENT TRAFFIC MANAGEMENT AND EMERGENCY ROUTING SYSTEM

FINAL PROJECT REPORT

Submitted in partial fulfillment of the requirements for the Object-Oriented Programming (OOP) Course

Submitted by:

- Student Name: **Santoshi swain**
- Roll Number:**B125113**
- Department: **Computer Science and Engineering**
- Semester: **3rd**
- College: **IIIT bhubaneswer**

Academic Year: 2026–2027

ABSTRACT

The **Intelligent Traffic Management and Emergency Routing System** is a C++-based console application designed to simulate basic traffic management and emergency vehicle routing.

Traffic congestion and road closures can cause delays in transportation, especially for emergency vehicles such as ambulances, fire engines, and police vehicles. Finding a suitable route under changing traffic conditions is an important challenge.

This project provides a simulated road network consisting of intersections connected through roads. It supports traffic signal management using vehicle counts from four directions, updating road traffic levels, closing and reopening roads, and finding the fastest available route.

The system uses Dijkstra's shortest-path algorithm to calculate routes based on estimated travel time. It considers different traffic levels and avoids roads that are marked as closed.

A priority queue is used to manage emergency vehicles according to their assigned priorities. Emergency vehicles can be added to the queue, displayed, and processed one at a time.

The project is implemented using C++ and demonstrates the practical application of classes, objects, encapsulation, graphs, priority queues, and algorithms.

TABLE OF CONTENTS

1. Introduction
2. Problem Statement
3. Objectives
4. Scope of the Project
5. Existing System
6. Proposed System
7. Software and Hardware Requirements
8. Technologies Used
9. Concepts Used
10. System Architecture
11. Methodology and Working
12. Algorithms and Data Structures
13. Module Description
14. Implementation Details
15. Sample Inputs and Outputs
16. Testing
17. Advantages
18. Limitations
19. Future Scope
20. Conclusion
21. References
22. Appendix – Source Code

---

1. INTRODUCTION

Traffic management is an important part of transportation systems. Increasing numbers of vehicles can cause traffic congestion, delays, and difficulties in reaching destinations.

Emergency vehicles such as ambulances, fire engines, and police vehicles require quick access to their destinations. Congested or closed roads can create additional delays during emergencies.

The Intelligent Traffic Management and Emergency Routing System is developed to simulate a system that can manage traffic information and help identify suitable routes.

The application represents a road network using intersections and roads. Each road stores information about its destination, distance, traffic level, and availability.

The system also provides traffic signal management, road traffic updates, road closure management, shortest-path calculation, and emergency vehicle prioritization.

The entire application is implemented in C++ using a menu-driven console interface.

2. PROBLEM STATEMENT

Traditional traffic management systems may use fixed signal timings and may not always respond effectively to changing traffic conditions.

Roads may become congested or unavailable because of accidents, construction, or other unexpected situations.

Emergency vehicles may experience delays when travelling through congested areas. Selecting a suitable route manually can also be difficult when several roads have different traffic conditions.

Therefore, there is a need for a software-based traffic management system that can:

- Manage traffic signal information using vehicle counts.
- Represent intersections and roads as a connected network.
- Update traffic levels on roads.
- Close and reopen roads.
- Calculate the fastest available route based on traffic conditions.
- Manage emergency vehicles according to their priority.

This project addresses these requirements through a C++ console-based simulation.

3. OBJECTIVES

The main objectives of this project are:

1. To develop a traffic management simulation using C++.
2. To manage traffic signal information based on vehicle counts from four directions.
3. To represent roads and intersections using a graph.
4. To implement Dijkstra's algorithm for fastest available route calculation.
5. To consider different traffic levels while calculating travel time.
6. To support road closure and reopening.
7. To manage emergency vehicles using a priority queue.
8. To process emergency vehicles according to their assigned priorities.
9. To apply object-oriented programming concepts in a practical project.
10. To understand the use of data structures and algorithms in transportation-related problems.

4. SCOPE OF THE PROJECT

The project focuses on simulating traffic management and emergency routing using a predefined road network.

The system provides the following features:

- Traffic signal management for North, South, East, and West directions.
- Displaying the road network.
- Updating road traffic levels.
- Closing and reopening roads.
- Finding the fastest available route between two intersections.
- Adding emergency vehicles.
- Displaying the emergency vehicle queue.
- Processing the next emergency vehicle according to priority.

The current implementation is a console-based simulation. It does not use live traffic sensors, GPS tracking, real-time maps, or physical traffic signals.

5. EXISTING SYSTEM

In conventional traffic management, traffic signal timings may be fixed or managed manually.

Route selection may also depend on manual decisions or basic distance calculations.

Some limitations of such approaches include:

- Difficulty adapting to changing traffic conditions.
- Delays caused by congested roads.
- Problems caused by road closures.
- Difficulty selecting routes for emergency vehicles.
- Lack of an integrated simulation for traffic and emergency management.

These challenges motivate the development of the proposed system.

6. PROPOSED SYSTEM

The proposed system is a C++ console application that combines traffic signal management, road network management, route calculation, and emergency vehicle prioritization.

It consists of three main classes:

TrafficSignal: Manages vehicle counts and green signal timings for four directions.

RoadNetwork: Maintains intersections and roads, updates traffic levels, manages road closures, and calculates the fastest available route.

EmergencyManager: Maintains emergency vehicle requests using a priority queue and processes the next vehicle according to priority.

The application provides a menu through which users can access all these features.

Main workflow

1. Start the application.
2. Initialize traffic signals, the road network, and the emergency manager.
3. Select an operation from the main menu.
4. Enter the required information.
5. Perform traffic or road management operations.
6. Calculate the fastest available route when required.
7. Add or process emergency vehicles.
8. Display the result in the console.
9. Exit the application.

7. SOFTWARE AND HARDWARE REQUIREMENTS

7.1 Software Requirements

Component	Requirement
Operating System	Windows
Programming Language	C++
Code Editor	Visual Studio Code
Compiler	G++
Compiler Standard	C++17

Interface	Console-based application
Data Structures	Graph, Vector, Priority Queue
Version Control	Git

7.2 Hardware Requirements

- Processor: Dual-core processor or higher.
- RAM: Minimum 4 GB recommended.
- Storage: Sufficient space for source files and compiler.
- Display: Standard laptop or desktop display.

8. TECHNOLOGIES USED

8.1 C++

C++ is the main programming language used to develop the project.

It provides support for classes, objects, encapsulation, functions, and standard data structures.

8.2 Standard Template Library (STL)

The project uses several C++ STL components:

- `vector` to store the adjacency list of the road network.
- `priority_queue` to implement shortest-path processing and emergency vehicle management.
- `pair` to store related values such as travel time and intersection number.
- `algorithm` to reverse the calculated route.
- `string` to store emergency vehicle types and display information.

8.3 GCC / G++

G++ is used to compile the C++ source files and generate the executable application.

9. CONCEPTS USED

9.1 Classes and Objects

A class is a user-defined data type that combines data members and member functions.

The project uses the following classes:

- `TrafficSignal`
- `RoadNetwork`
- `EmergencyManager`
- `EmergencyVehicle` (used to represent emergency vehicle information)

Objects are created in `main()` to access the functionality of these classes.

For example:

```
TrafficSignal signal;
RoadNetwork network(5);
EmergencyManager emergencyManager;
```

These objects manage the different parts of the application.

9.2 Encapsulation

Encapsulation means combining data and the functions that operate on that data inside a class.

For example, the `TrafficSignal` class stores vehicle counts and green signal timings as private data members.

Similarly, the `RoadNetwork` class manages road information through its member functions.

This helps organize the program and control access to internal data.

9.3 Abstraction

Abstraction means exposing necessary operations while hiding internal implementation details.

For example, the user can call:

```
network.shortestPath(source, destination);
```

The user does not need to understand the internal implementation of Dijkstra's algorithm to calculate a route.

9.4 Graph Data Structure

A graph consists of vertices and edges.

In this project:

- Vertices represent intersections.
- Edges represent roads connecting intersections.

The graph is stored using an adjacency list.

9.5 Priority Queue

A priority queue is used in two parts of the project:

1. Dijkstra's algorithm uses a priority queue to select the intersection with the smallest known travel time.
2. Emergency management uses a priority queue to process vehicles according to their assigned priority.

9.6 Functions and Modular Programming

The application is divided into different functions and classes. Each function performs a specific operation, such as updating traffic, displaying roads, calculating routes, or processing emergency vehicles.

This improves code organization and makes the application easier to understand and maintain.

Note: Inheritance and runtime polymorphism are not claimed as implemented features of this submitted version because they are not demonstrated in the source files provided for this report.

10. SYSTEM ARCHITECTURE

The project follows a modular, class-based architecture.

10.1 Traffic Signal Module

Responsible for:

- Storing vehicle counts.
- Updating vehicle counts.
- Calculating green signal timings.
- Displaying signal information.

10.2 Road Network Module

Responsible for:

- Creating intersections and roads.
- Maintaining road information.
- Updating traffic levels.
- Closing and reopening roads.
- Calculating the fastest available route.

10.3 Emergency Vehicle Module

Responsible for:

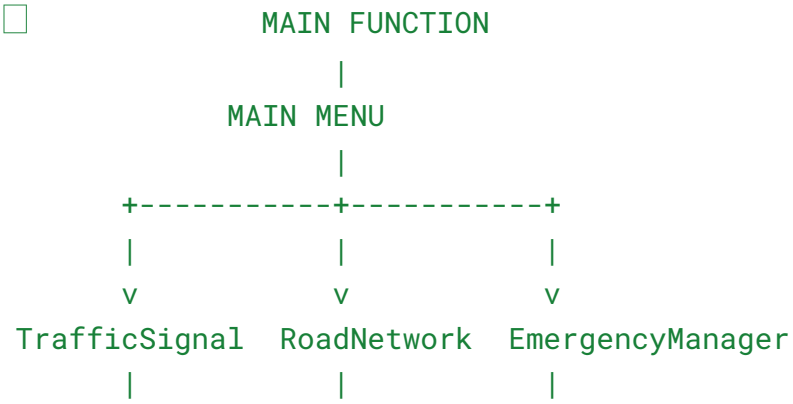
- Storing emergency vehicle details.
- Adding vehicles to the priority queue.
- Displaying the emergency queue.
- Processing the next emergency vehicle.

10.4 Main Control Module

The `main.cpp` file controls the complete application through a menu-driven interface.

It creates objects, accepts user input, and calls the appropriate class functions.

Architecture Diagram



```
      v           v           v
Vehicle Counts  Road Graph   Priority Queue
      |           |           |
      v           v           v
Signal Timing  Dijkstra     Emergency
Calculation    Algorithm    Processing
      |           |           |
      +-----------+-----------+
                  |
                  v
            CONSOLE OUTPUT
```

11. METHODOLOGY AND WORKING

11.1 Initialization of the System

When the application starts, three main objects are created:

```
TrafficSignal signal;
RoadNetwork network(5);
EmergencyManager emergencyManager;
```

The road network is initialized with five intersections.

The `RoadNetwork` constructor creates the roads connecting the intersections.

11.2 Road Network Creation

The project contains five intersections numbered from 0 to 4.

Seven undirected roads are created between these intersections.

Road	Connected Intersections	Distance
1	0 ↔ 1	4 km
2	0 ↔ 2	2 km

3	1 ↔ 2	1 km
4	1 ↔ 3	5 km
5	2 ↔ 3	8 km
6	2 ↔ 4	10 km
7	3 ↔ 4	2 km

Each road contains four important properties:

- Destination intersection.
- Distance.
- Traffic level.
- Road closure status.

All roads are open initially and have the default traffic level of 1, representing low traffic.

11.3 Traffic Signal Management

The TrafficSignal class manages traffic counts from four directions:

1. North
2. South
3. East
4. West

The user selects the traffic signal management option from the menu and enters the number of vehicles waiting in each direction.

Negative vehicle counts are rejected by the main program.

The class provides the following functions:

- `setVehicleCount()`
- `calculateGreenTime()`
- `displaySignal()`

These functions manage vehicle counts, calculate green signal timings, and display the signal information.

The exact green-time calculation rule is defined in the TrafficSignal implementation file.

11.4 Display Road Network

The user can select the option to display the road network.

The `displayRoads()` function displays:

- Starting intersection.
- Destination intersection.
- Road distance.
- Traffic level.
- Road status.

The road status is displayed as either OPEN or CLOSED.

Since roads are undirected, the display function prints each connection only once.

11.5 Updating Road Traffic Level

Each road supports three traffic levels:

Level	Description
1	Low
2	Medium
3	High

The user enters the two connected intersections and selects the required traffic level.

The `setTrafficLevel()` function updates the traffic level in both directions.

If the intersection values or traffic level are invalid, an error message is displayed.

If no road connects the selected intersections, the program reports that no such road exists.

11.6 Closing and Reopening Roads

The system allows the user to close or reopen a road.

The user enters the two intersections connected by the road and selects one of the following operations:

- Close road.
- Reopen road.

The `setRoadClosure()` function updates the closure status in both directions.

A closed road is not considered by the shortest-path algorithm.

This feature simulates situations where a road becomes unavailable because of an accident, construction, or other blockage.

11.7 Fastest Available Route Calculation

The system uses Dijkstra's algorithm to calculate the fastest available route between two intersections.

The user enters a source intersection and a destination intersection.

The algorithm considers:

- Road distance.
- Traffic level.
- Road availability.

The algorithm ignores closed roads and calculates the estimated travel time for the available roads.

If a valid route is found, the system displays the route and estimated travel time.

If no route exists, the system displays an appropriate message.

11.8 Emergency Vehicle Management

The system supports three types of emergency vehicles:

1. Ambulance
2. Fire Engine
3. Police Vehicle

Each vehicle is assigned a priority:

Vehicle	Priority

Ambulance	1
Fire Engine	2
Police Vehicle	3

The user enters the vehicle type, current intersection, and destination intersection.

The `addVehicle()` function creates an emergency vehicle object and inserts it into the priority queue.

The `displayQueue()` function displays the emergency vehicles without removing them from the original queue.

The `processNext()` function removes the next vehicle from the queue and calls the road network's `shortestPath()` function to calculate its route.

The lower priority number represents a higher priority in the intended queue ordering.

12. ALGORITHMS AND DATA STRUCTURES

12.1 Graph Representation Using Adjacency List

The road network is represented using an adjacency list.

Each intersection stores a list of roads connected to it.

This representation is useful because the system needs to access the neighboring intersections while calculating routes.

Since roads are undirected, each road is stored in both directions.

12.2 Dijkstra's Shortest-Path Algorithm

Dijkstra's algorithm is used to calculate the minimum travel-time route between two intersections.

Instead of considering only road distance, the project calculates the estimated travel time using the traffic level.

The assumed speeds are:

Traffic Level	Assumed Speed
Low	40 km/h
Medium	25 km/h
High	10 km/h

The travel time of a road is calculated using:

Travel Time = Distance / Speed

The algorithm works as follows:

1. Validate the source and destination intersections.
2. Initialize the travel time of the source to zero.
3. Initialize the travel times of all other intersections to infinity.
4. Insert the source intersection into a priority queue.
5. Extract the intersection with the smallest known travel time.
6. Check all roads connected to that intersection.
7. Skip roads that are closed.
8. Calculate the travel time for each available road.
9. Update the travel time if a faster route is found.
10. Store the previous intersection to reconstruct the route.
11. Continue until the priority queue becomes empty.
12. Reconstruct and display the fastest available route.

The final estimated travel time is converted into minutes by multiplying the calculated hours by 60.

Time Complexity

For a graph represented using an adjacency list and a priority queue, the typical time complexity is:

O((V + E) log V)

Where:

- V represents the number of intersections.
- E represents the number of roads.

12.3 Priority Queue for Emergency Vehicles

A priority queue is used to manage emergency vehicle requests.

Each emergency vehicle contains:

- Vehicle type.
- Current intersection.
- Destination intersection.
- Priority value.

The queue selects vehicles according to the comparison rule defined in the `ComparePriority` structure.

The intended priority order is:

Ambulance → Fire Engine → Police Vehicle.

The priority queue helps process more important emergency requests before lower-priority requests.

13. MODULE DESCRIPTION

13.1 Traffic Signal Module

Files: `TrafficSignal.h`, `trafficSignal.cpp`

This module manages traffic signal information.

It stores four vehicle counts and four green signal timings.

Functions:

- `TrafficSignal()`
- `setVehicleCount()`
- `calculateGreenTime()`
- `displaySignal()`

13.2 Road Network Module

Files: `RoadNetwork.h`, `roadNetwork.cpp`

This module manages the road network.

Functions:

- `RoadNetwork()`
- `addRoad()`
- `displayRoads()`
- `setTrafficLevel()`
- `setRoadClosure()`
- `shortestPath()`

13.3 Emergency Vehicle Module

Files: `EV.h`, `EV.cpp`

This module manages emergency vehicle requests.

Functions:

- `addVehicle()`
- `processNext()`
- `displayQueue()`

The module uses a priority queue to organize emergency vehicles.

13.4 Main Module

File: `main.cpp`

This module controls the application.

It displays the main menu, accepts input, validates selected options, and calls the appropriate class functions.

14. IMPLEMENTATION DETAILS

The project is divided into separate header and implementation files.

Project Folder Structure

```
Intelligent-Traffic-Management/
│
├── main.cpp
│
├── TrafficSignal.h
├── trafficSignal.cpp
│
├── RoadNetwork.h
├── roadNetwork.cpp
│
├── EV.h
├── EV.cpp
│
└── executable file
```

Header files contain class declarations, while implementation files contain their member function definitions.

This separation makes the program modular and easier to maintain.

Compilation

The project can be compiled using:

```
g++ main.cpp trafficSignal.cpp roadNetwork.cpp EV.cpp -o traffic
```

Execution

On Windows PowerShell:

```
.\traffic.exe
```

After execution, the main menu is displayed.

15. SAMPLE INPUTS AND OUTPUTS

The following examples illustrate the expected console interaction. Add screenshots of your actual execution to the final printed report.

15.1 Main Menu

```
=====  INTELLIGENT TRAFFIC MANAGEMENT
AND EMERGENCY ROUTING SYSTEM =====

1. Manage Traffic Signals
2. Display Road Network
3. Update Road Traffic Level
4. Close/Reopen a Road
5. Find Fastest Available Route
6. Add Emergency Vehicle
7. Process Next Emergency Vehicle
8. Display Emergency Queue
9. Exit

Enter choice:
```

15.2 Display Road Network

Input:

```
Enter choice: 2
```

Expected output:

```
--- Road Network ---

0 <-> 1 | Distance: 4 km | Traffic: Low | Status: OPEN
0 <-> 2 | Distance: 2 km | Traffic: Low | Status: OPEN
1 <-> 2 | Distance: 1 km | Traffic: Low | Status: OPEN
1 <-> 3 | Distance: 5 km | Traffic: Low | Status: OPEN
2 <-> 3 | Distance: 8 km | Traffic: Low | Status: OPEN
2 <-> 4 | Distance: 10 km | Traffic: Low | Status: OPEN
3 <-> 4 | Distance: 2 km | Traffic: Low | Status: OPEN
```

15.3 Update Traffic Level

Example input:

```
Enter choice: 3
Enter first intersection (0-4): 0
Enter second intersection (0-4): 1

1. Low traffic
2. Medium traffic
3. High traffic

Enter traffic level: 2
```

Expected output:

```
Traffic level updated successfully.
```

15.4 Close a Road

Example input:

```
Enter choice: 4
Enter first intersection (0-4): 0
Enter second intersection (0-4): 1

1. Close road
2. Reopen road

Enter choice: 1
```

Expected output:

```
Road closed successfully.
```

15.5 Find Fastest Available Route

Example input:

```
Enter choice: 5
Enter source intersection (0-4): 0
```

```
Enter destination intersection (0-4): 4
```

When all roads have their default low traffic level and are open, the expected fastest route is:

```
Fastest available route: 0 -> 2 -> 1 -> 3 -> 4

Estimated travel time: 15 minutes
```

This result uses the assumed low-traffic speed of 40 km/h for each road.

15.6 Add Emergency Vehicle

Example input:

```
Enter choice: 6

1. Ambulance
2. Fire Engine
3. Police Vehicle

Select vehicle type: 1
Enter current intersection (0-4): 0
Enter destination (0-4): 4
```

Expected output:

```
Ambulance added to emergency queue.
```

15.7 Process Emergency Vehicle

Example output:

```
=== EMERGENCY ROUTING ===

Vehicle: Ambulance
Starting intersection: 0
Destination: 4

Finding the fastest available route...

Fastest available route: 0 -> 2 -> 1 -> 3 -> 4
```

```
Estimated travel time: 15 minutes

Emergency routing completed successfully.
```

15.8 Exit the System

Input:

```
Enter choice: 9
```

Output:

```
Exiting system. Goodbye!
```

16. TESTING

The following test cases can be used to verify the functionality of the application.

Test Case	Input / Operation	Expected Result
1	Start the program	Main menu is displayed
2	Select option 2	Road network is displayed
3	Enter a valid traffic level	Traffic level is updated
4	Enter an invalid traffic level	Error message is displayed
5	Close an existing road	Road status becomes CLOSED
6	Reopen a closed road	Road status becomes OPEN
7	Select valid source and destination	Fastest available route is displayed
8	Enter an invalid intersection	Error message is displayed

9	Add an ambulance	Ambulance is added to the queue
10	Display emergency queue	Queued vehicles are displayed
11	Process next emergency vehicle	Highest-priority queued vehicle is processed
12	Process an empty queue	Empty-queue message is displayed

17. ADVANTAGES

- Provides a simple traffic management simulation.
- Supports traffic signal management.
- Uses a graph to represent road connections.
- Calculates routes using estimated travel time.
- Considers different traffic levels.
- Avoids roads marked as closed.
- Supports road closure and reopening.
- Uses a priority queue for emergency vehicle management.
- Uses modular C++ classes.
- Demonstrates practical application of data structures and algorithms.

18. LIMITATIONS

- The road network contains only five predefined intersections.
- Vehicle counts are entered manually.
- Traffic information is not collected from real sensors.
- The application does not connect to physical traffic signals.
- The system does not use real-time GPS tracking.
- Traffic speed values are assumed for simulation.
- The system is a console-based application.
- The project does not use live road maps or external traffic information.

19. FUTURE SCOPE

The project can be extended in the future through the following improvements:

1. Add more intersections and roads to create a larger network.

2. Integrate real-time traffic information.
3. Use GPS to track emergency vehicles.
4. Add automatic traffic signal control.
5. Introduce machine learning for traffic prediction.
6. Add accident detection and automatic road closure information.
7. Develop a graphical user interface.
8. Store traffic information and historical data in a database.
9. Integrate real geographical maps.
10. Provide emergency vehicle route assistance using live location data.

20. CONCLUSION

The Intelligent Traffic Management and Emergency Routing System demonstrates how C++ programming, object-oriented concepts, data structures, and graph algorithms can be applied to a practical transportation-related problem.

The system provides traffic signal management, road network display, traffic-level updates, road closure management, and fastest available route calculation.

Dijkstra's algorithm is used to calculate routes based on estimated travel time, considering both road distance and traffic levels. Closed roads are excluded from route calculation.

The EmergencyManager class uses a priority queue to organize emergency vehicle requests and process them according to priority.

Through this project, I gained practical knowledge of classes, objects, encapsulation, abstraction, graphs, adjacency lists, priority queues, and shortest-path algorithms.

The project provides a foundation for understanding how software-based traffic management systems can be developed and improved in the future.

21. REFERENCES

1. Bjarne Stroustrup, *The C++ Programming Language*.
2. C++ Reference: https://en.cppreference.com/
3. GeeksforGeeks – Graph Data Structure and Dijkstra's Algorithm.
4. C++ Standard Template Library documentation.
5. Object-Oriented Programming course materials.
6. Data Structures and Algorithms course materials.

22. APPENDIX – SOURCE CODE

The complete source code should be attached in this section.

Include the following files in their original form:

1. `main.cpp`
2. `TrafficSignal.h`
3. `trafficSignal.cpp`
4. `RoadNetwork.h`
5. `roadNetwork.cpp`
6. `EV.h`
7. `EV.cpp`

The source code appendix should contain the actual program files used to compile and execute the project.

The code in the report must match the final code submitted to the faculty.
